appendix A, section A.1.1, for more on move semantics). Such a transfer leaves the source object with a `NULL` pointer. This moving of values allows objects of this type to be accepted as function parameters or returned from functions. Where the source object is a temporary, the move is automatic, but where the source is a named value, the transfer must be requested directly by invoking `std::move()`. The following example shows the use of `std::move` to transfer ownership of a dynamic object into a thread:

```
void process_big_object(std::unique_ptr<big_object>);

std::unique_ptr<big_object> p(new big_object);
p->prepare_data(42);
std::thread t(process_big_object,std::move(p));
```

By specifying `std::move(p)` in the `std::thread` constructor, the ownership of the `big_object` is transferred first into internal storage for the newly created thread and then into `process_big_object`.

Several of the classes in the Standard Thread Library exhibit the same ownership semantics as `std::unique_ptr`, and `std::thread` is one of them. Though `std::thread` instances don't own a dynamic object in the same way as `std::unique_ptr` does, they do own a resource: each instance is responsible for managing a thread of execution. This ownership can be transferred between instances, because instances of `std::thread` are *movable*, even though they aren't *copyable*. This ensures that only one object is associated with a particular thread of execution at any one time while allowing programmers the option of transferring that ownership between objects.

2.3. Transferring ownership of a thread

Suppose you want to write a function that creates a thread to run in the background but passes back ownership of the new thread to the calling function rather than waiting for it to complete, or maybe you want to do the reverse: create a thread and pass ownership in to some function that should wait for it to complete. In either case, you need to transfer ownership from one place to another.